Hackathon Plan

Cloud Native Hackathon · Monday, November 9, 2026 · The Shop Workspace, Salt Lake City

Tracks: Elasticsearch + Gitar + Merge Forward

The app

A website that runs locally for the demo, built with **Python + FastAPI**, **plain HTML/CSS/JS with SVG floor plans**, and **Elasticsearch** (local start-local) holding all 582 KubeCon talks.

What a user can do:

1. **Search and filter** by keyword, day (Mon–Thu), topic, registration type and wheelchair access.
2. **Pick a Salt Palace level** and see talk counts per level as a simple heatmap.
3. **See the Level 2 floor plan** with rooms colored by how many matching talks they have. Click a room for a popup of its talks.
4. **Add talks to My Schedule** (Mon–Thu), with **conflicts highlighted in red**.
5. **Hyatt and off-site talks** appear as lists. Off-site ones get an "accessibility unverified" badge.

Tech stack

Part	Choice
Backend	Python 3.11+, FastAPI, the `elasticsearch` client
Frontend	One `index.html`, `app.js` and `style.css`, plus hand-drawn SVG floor plans
Data	Elasticsearch index `kubecon-talks`, loaded from `talks.bulk.ndjson`
Personal schedule	Browser `localStorage`: no logins, no personal data
Code review	GitHub (public repo) + Gitar + GitHub Actions running `pytest`

Repo layout

```
app/main.py          # FastAPI: /search endpoint + serves frontend
app/search.py        # builds the Elasticsearch query (filters + aggregations)
app/schedule.py      # conflict checker (pure function, easy to test)
static/              # index.html, app.js, style.css, floorplans/*.svg
tests/               # pytest for schedule.py and search.py query building
data/load.py         # loads dataset into Elasticsearch (data files gitignored)
.gitar/review/       # Gitar review instructions
.github/workflows/ci.yml
README.md
```

Track 1: Elasticsearch, the core data layer

Every screen calls a single endpoint, `GET /search?q=&day=&topic=&registration=&wheelchair=&venue=&level=`. It sends one Elasticsearch query that returns **both** the matching talks **and** these aggregations:

UI feature	Elasticsearch feature
Search box	`multi_match` on `title^3`, `abstract`, `speakers.name`, with `fuzziness: "AUTO"` (handles typos)
Filters	`bool.filter` with `term` on day, `topics`/`topics_inferred`, `registration.type`, `accessibility.wheelchair_accessible`
Venue counts on home	`terms` aggregation on `location.venue`
Level heatmap	`terms` aggregation on `location.level`
Room heatmap	`terms` aggregation on `location.room`
Talks-per-hour chart (stretch)	`date_histogram` on `start`, 1-hour interval
Autocomplete (stretch)	`search_as_you_type` on `title.suggest` (already in the mapping)

What to say in the demo: "Every click is one Elasticsearch query. Full-text search narrows the talks, and the same query's aggregations color the map."

Track 2: Gitar, never merge broken

- **First 15 minutes:** create the public repo and install the Gitar GitHub App. Turn on the free trial so you get auto-apply and rules. Protect `main` so nobody pushes to it directly.
- **Every feature** goes branch → PR → Gitar review → teammate approves → merge. Aim for 8–12 PRs over the day.
- **`.gitar/review/` instructions** in plain English, matched to your project. For example:
  - "All session times are Mountain Time. Never compare times as strings."
  - "Never commit Elasticsearch API keys or the data files in `data/`."
  - "Every clickable SVG room must have an `aria-label` and be keyboard focusable."
  - "Search endpoint must handle empty queries and unknown filter values."
- **CI** is a GitHub Action that runs `pytest`, with tests on `schedule.py` (conflict detection) and the query builder.
- **Demo moment:** a PR where Gitar caught a real bug, plus one CI failure that Gitar diagnosed and **auto-applied** a fix for. If none happens naturally, push a PR around 4 PM that breaks a test.

Track 3: Merge Forward, accessibility

Build these in as features, not afterthoughts:

- **Wheelchair filter.** Official venues are marked accessible, citing the KubeCon accessibility page. Off-site venues show **"unverified, contact host"** instead of a guess.
- **Quiet rooms** (Salt Palace 260A and 260B) marked on the Level 2 map.
- **A "Beginner" audience filter**, so newcomers can find approachable talks.
- **The map is accessible:** every room has a label, you can tab through rooms with the keyboard, colors meet contrast standards, and counts are shown as **numbers too**, not color alone.
- **A text-size and high-contrast toggle.**
- **Demo pitch:** "Planning a conference is harder if you use a wheelchair, get overwhelmed in crowds, or are new. This shows where to go, how to get in, and where to rest."

Who does what

Person A: data and backend	Person B: frontend and accessibility
Load the dataset into Elasticsearch	Page layout, filters, search box
`/search` endpoint: query, filters, aggregations	Level 2 SVG floor plan (then Level 1)
`schedule.py` conflict checker + pytest	Heatmap coloring + room popups
GitHub Actions CI	My Schedule panel (`localStorage`)
README: setup + architecture	Accessibility features + Gitar review rules

You review each other's PRs alongside Gitar, so both of you understand all the code for judges' questions.

Timeline for Monday

Time	Person A	Person B
9:00–10:00	Check in, set up	Check in, set up
10:00–11:00	Kickoff. Ask organizers if you can use the public Sessionize data.	Kickoff. Create the repo, install Gitar, write `.gitar/review/` rules.
11:00–12:00	Elasticsearch running, data loaded, `/search` returns talks	`index.html` skeleton, filter controls, search box
12:00–1:00	Aggregations working, CI + first tests	Talk list connected to `/search`, start the Level 2 SVG
1:00–2:30	Lunch and Kelsey panel (light work if you want)	Lunch and panel
2:30–3:30	Conflict checker + tests, help with integration	Room heatmap coloring + click popups
3:30–4:15	Stretch: hourly chart or autocomplete	My Schedule panel + accessibility toggles
4:15–4:45	**Feature freeze.** Gitar demo PR (CI break → auto-apply). Bug fixes.	Bug fixes, Level 1 SVG if time allows
4:45–5:15	README: setup, architecture, data source, **AI disclosure**	Practice the 3-minute demo, take screenshots
5:15	**Submit** (15 minutes of buffer before 5:30)	

If you fall behind: cut Level 1, then the hourly chart, then the text-size toggle. **Never cut** search, filters, the Level 2 heatmap, or My Schedule.

Demo script (about 3 minutes)

1. **The problem (20 seconds):** 582 talks, multiple buildings, and it's hard to plan, especially with accessibility needs.
2. **Search:** type "observabilty" with the typo. Fuzzy search still finds it.
3. **Filter:** Tuesday + AI + wheelchair accessible. Show the level counts and room heatmap updating, then click a room for the popup.
4. **Schedule:** add two overlapping talks and show the red conflict warning.

5. **Accessibility:** quiet rooms, keyboard navigation, the "unverified" badge.
6. **Gitar:** show PR history, a bug Gitar caught, and the auto-apply fix.
7. **Close:** "One Elasticsearch query powers every view."

Rules checklist

- ☐ No code or design work before 11:00. Paper sketches tonight are fine.
- ☐ Public repo, with a README containing setup instructions
- ☐ **AI disclosure in the README**, covering the pre-built dataset script and any assistant use
- ☐ Both of you can explain every file and design choice
- ☐ Data permission asked at kickoff. The data files are gitignored and loaded by `data/load.py`.
- ☐ No secrets in the repo (use a `.env` file and gitignore it)
- ☐ One project, submitted before 5:30

Tonight's prep

- ☐ Both of you: Gitar account, GitHub account, Docker, Python 3.11+, `pip install fastapi uvicorn elasticsearch pytest`
- ☐ Run `start-local` and confirm Kibana opens
- ☐ Load `C:\Users\keato\kubecon-dataset\out\talks.bulk.ndjson` and try a search plus an aggregation in Kibana Dev Tools (this counts as research)
- ☐ Paper-sketch Salt Palace Levels 1 and 2 using the public floor plan as a reference
- ☐ Confirm the A/B split and agree on a PR naming convention, like `feat/search-endpoint`